

PROJECT SUBMISSION REPORT

Distributed Job Scheduler — Production-Grade Platform

Vansh Agrawal

Registration No: RA2311026010120

GitHub: github.com/vansh070605/distributed-job-scheduler

Submission Date: July 2, 2026

1. Executive Summary

This project implements a complete, production-quality, distributed job scheduling platform capable of executing asynchronous background jobs — immediate, delayed, scheduled, recurring, and batch runs — across multiple worker nodes.

Built using Clean Architecture principles, the platform prioritizes reliability, concurrency guarantees, and system observability over a simple CRUD setup. The technology stack consists of a FastAPI REST API server, a Celery worker execution pool, a Celery Beat scheduler, a Redis broker and lock manager, PostgreSQL 15 for persistent storage, and a React + Vite + TypeScript SaaS dashboard.

2. System Architecture

The platform separates concerns across three independent layers: the API layer, the scheduling engine, and the execution pool.

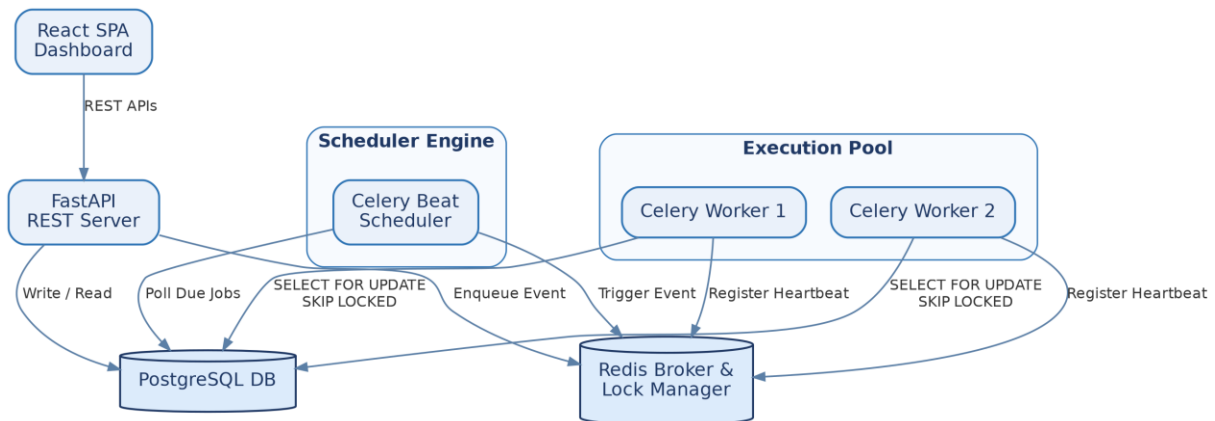


Figure 1 — High-Level Architecture

2.1 Separation of Concerns

- FastAPI REST API — handles user authentication, configures queue variables, and ingests job payloads.
- Celery Beat (Scheduler) — determines only when jobs are due; polls PostgreSQL for scheduled and recurring tasks, flags them as queued, and pushes execution events to Redis.
- Celery Workers — responsible only for executing jobs; pull trigger events from Redis, lock rows in PostgreSQL, process payloads, update stats, and write console logs.

3. Database Design & Entity Relationship

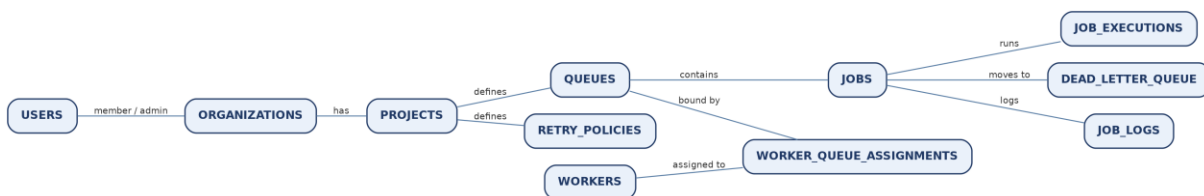


Figure 2 — Entity Relationship Diagram

3.1 Relational Table Design & Optimization

- Users — primary key (UUID), indexed on email; stores password hashes and role levels.
- Jobs — custom compound index `idx_jobs_claim` on (status, queue_id, priority_override, created_at) to support ultra-fast claiming. Holds `idempotency_key` and `last_worker_id`.
- Queue Metrics (CQRS-lite) — pre-aggregated statistics (queued, running, throughput, averages) computed asynchronously by Celery Beat every 10 seconds, avoiding heavy, lock-inducing dashboard queries over millions of executions.
- Worker Assignments — normalizes many-to-many worker-to-queue bindings.

4. Key Technical Implementations

4.1 Concurrency Protection — Atomic Claims

To prevent duplicate execution when multiple worker processes poll the same queue, jobs are claimed using database pessimistic row-level locking (FOR UPDATE SKIP LOCKED):

[app/repositories/job.py](#)

```
subquery = (
    select(Job.id)
    .where(and_(Job.status == "queued", Job.queue_id == queue_id))
    .order_by(
        Job.priority_override.desc().nullslast(),
        Job.created_at.asc(),
        Job.id.asc()
    )
    .limit(1)
    .with_for_update(skip_locked=True)
)
```

4.2 Idempotency Key Handling

Prior to payload execution, workers check whether the unique `idempotency_key` has already been run successfully under the project scope:

[app/workers/tasks.py](#)

```
completed_job = (
    session.query(Job)
    .filter(
        Job.project_id == job.project_id,
        Job.idempotency_key == job.idempotency_key,
        Job.status == "completed",
        Job.id != job.id
    )
    .first()
)
```

5. Automated Testing & Verification Logs

Automated integration tests were implemented to verify critical backend functions:

- JWT token cryptography and password hashing contexts.
- Exponential, linear, and fixed delay retry mathematics.
- REST API endpoint security and validation status codes.

Pytest Execution Output

Test	Result
test_security.py :: test_password_hashing	PASSED
test_security.py :: test_token_creation_and_validation	PASSED
test_security.py :: test_retry_delay_calculations	PASSED
test_api.py :: test_health_check_endpoint	PASSED
test_api.py :: test_auth_registration	PASSED

Result: 5 passed in 0.86s

6. How to Deploy & Verify Locally

6.1 Clone & Spin Up Containers

```
git clone https://github.com/vansh070605/distributed-job-scheduler.git
cd distributed-job-scheduler
docker compose up --build
```

6.2 Access URLs

- SaaS Dashboard — <http://localhost:5173>
- API Docs (Swagger) — <http://localhost:8000/docs>

6.3 Bootstrap Verification

- Register an admin user.
- Click the “Bootstrap Test Environment” button to auto-seed projects and default queues.
- Submit immediately-running or delayed/cron jobs, and inspect runs, heartbeat checks, and logs in the console drawer.